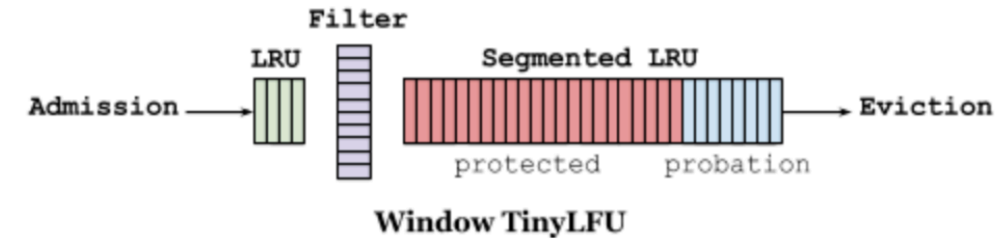
caffeine 源码全解析

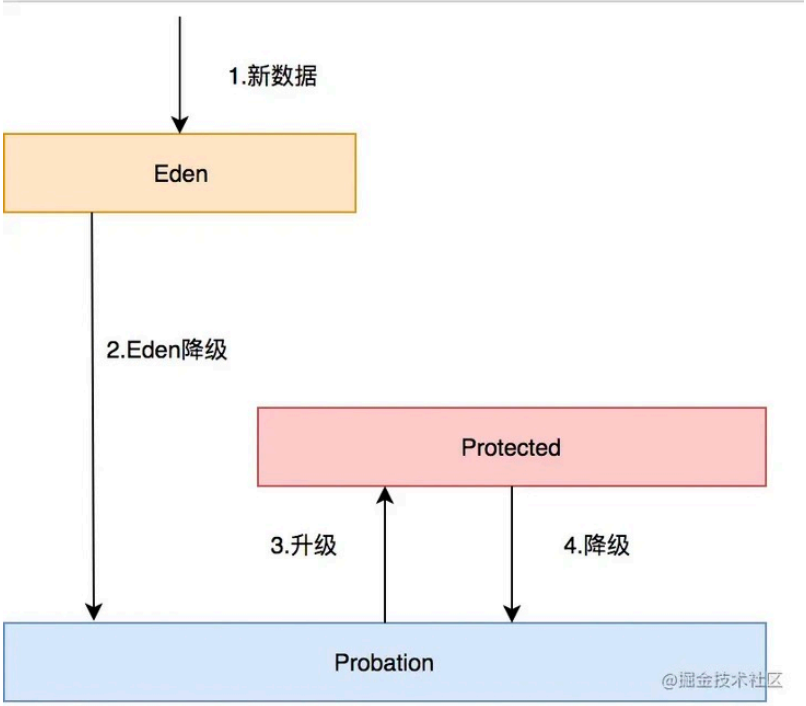
由 朱明创建, 最后修改于七月 14, 2022 45 views since 12 Jul 2022

一、caffeine高性能设计--淘汰策略

Caffeine 通过测试发现 TinyLFU 在面对突发性的稀疏流量（sparse bursts）时表现很差，因为新的记录（new items）还没来得及建立足够的频率就被剔除出去了，这就使得命中率下降。于是 Caffeine 设计出一种新的 policy，即 Window Tiny LFU（W-TinyLFU），并通过实验和实践发现 W-TinyLFU 比 TinyLFU 表现的更好。

下图为caffeine的缓存淘汰结构





接下来通过caffeine的源码来分析caffeine的淘汰策略设计，下面为所有对cache操作都需要执行的cache状态管理流程。

```
@GuardedBy("evictionLock")
void maintenance(@Nullable Runnable task) {
    this.lazySetDrainStatus(2);

    try {
        this.drainReadBuffer();
        this.drainWriteBuffer();
        if (task != null) {
            task.run();
        }

        this.drainKeyReferences();
        this.drainValueReferences();
        this.expireEntries();
        //把符合条件的记录淘汰掉
        this.evictEntries();
        //动态调整window区和protected区的大小
        this.climb();
    } finally {
        if (this.drainStatus() != 2 || !this.casDrainStatus(2, 0)) {
            this.lazySetDrainStatus(1);
        }

    }
```

```
}
```

接下来说下目前有关淘汰策略的数据结构，主要是三个分区的内部情况。

```
//最大的个数限制
long maximum;
//当前的个数
long weightedSize;
//window区的最大限制
long windowMaximum;
//window区当前的个数
long windowWeightedSize;
//protected区的最大限制
long mainProtectedMaximum;
//protected区当前的个数
long mainProtectedWeightedSize;
//下一次需要调整的大小（还需要进一步计算）
double stepSize;
//window区需要调整的大小
long adjustment;
//命中计数
int hitsInSample;
//不命中的计数
int missesInSample;
//上一次的缓存命中率
double previousSampleHitRate;

final FrequencySketch<K> sketch;
//window区的LRU queue（FIFO）
final AccessOrderDeque<Node<K, V>> accessOrderWindowDeque;
//probation区的LRU queue（FIFO）
final AccessOrderDeque<Node<K, V>> accessOrderProbationDeque;
//protected区的LRU queue（FIFO）
final AccessOrderDeque<Node<K, V>> accessOrderProtectedDeque;
```

下面代码为淘汰数据的第一步，主要与window数据在满size后的处理流程相关

```
/** Evicts entries if the cache exceeds the maximum. */
 @GuardedBy("evictionLock")
 void evictEntries() {
   if (!evicts()) {
     return;
   }
   //淘汰window区的数据的条数
   int candidates = evictFromWindow();
   //淘汰Main区也就是protect和probation区域的的数据
   evictFromMain(candidates);
 }

 /**
  * Evicts entries from the window space into the main space while the window size exceeds a
  * maximum.
  *
  * @return the number of candidate entries evicted from the window space
  */
 @GuardedBy("evictionLock")
 int evictFromWindow() {
   int candidates = 0;
   //获取window queue的头部节点
   Node<K, V> node = accessOrderWindowDeque().peek();
   //如果window区超过了最大的限制，那么就要把"多出来"的记录做处理
   while (windowWeightedSize() > windowMaximum()) {
     // The pending operations will adjust the size to reflect the correct weight
     if (node == null) {
       break;
     }

     Node<K, V> next = node.getNextInAccessOrder();
     if (node.getPolicyWeight() != 0) {
       //把node定位在probation区
       node.makeMainProbation();
       /从window区去掉该节点
       accessOrderWindowDeque().remove(node);
       //加入到probation queue，相当于把节点移动到probation区
       accessOrderProbationDeque().add(node);
       candidates++;
```

```
        //因为移除了一个节点，所以需要调整window的size
        setWindowWeightedSize(windowWeightedSize() - node.getPolicyWeight());
      }
      //处理该节点下一个阶段，node一直是头节点
      node = next;
    }
    //最终返回windows区域淘汰的数量
    return candidates;
  }
```

下面代码为最复杂和关键的部分，是淘汰数据的具体过程。

最终竞争candidate和victim是最后的流程，本质上因为caffeine的内部有很多其它影响因素，希望能够在之前将内部“无用”的数据淘汰，例如gc淘汰、本身权重为0或者过大等数据淘汰。最终没有办法进行竞争，将实际数据淘汰删除。

```
/**
   * Evicts entries from the main space if the cache exceeds the maximum capacity. The main space
   * determines whether admitting an entry (coming from the window space) is preferable to retaining
   * the eviction policy's victim. This is decision is made using a frequency filter so that the
   * least frequently used entry is removed.
   *
   * The window space candidates were previously placed in the MRU position and the eviction
   * policy's victim is at the LRU position. The two ends of the queue are evaluated while an
   * eviction is required. The number of remaining candidates is provided and decremented on
   * eviction, so that when there are no more candidates the victim is evicted.
   *
   * @param candidates the number of candidate entries evicted from the window space
   */
  @GuardedBy("evictionLock")
  void evictFromMain(int candidates) {
    int victimQueue = PROBATION;
    //首先victim是probation queue的头部
    Node<K, V> victim = accessOrderProbationDeque().peekFirst();
    //candidate是probation queue的尾部，也就是刚从window晋升来的
    Node<K, V> candidate = accessOrderProbationDeque().peekLast();
    //当cache不够容量时才做处理
    while (weightedSize() > maximum()) {
      // Stop trying to evict candidates and always prefer the victim
      if (candidates == 0) {
        candidate = null;
      }

      // Try evicting from the protected and window queues
      // 在probation队列为空，按先protected后window原则淘汰数据。
      if ((candidate == null) && (victim == null)) {
        if (victimQueue == PROBATION) {
          victim = accessOrderProtectedDeque().peekFirst();
          victimQueue = PROTECTED;
          continue;
        } else if (victimQueue == PROTECTED) {
          victim = accessOrderWindowDeque().peekFirst();
          victimQueue = WINDOW;
          continue;
        }

        // The pending operations will adjust the size to reflect the correct weight
        break;
      }

      // Skip over entries with zero weight
      // 如果该node的权重为0，默认其可以被直接删除
      if ((victim != null) && (victim.getPolicyWeight() == 0)) {
        victim = victim.getNextInAccessOrder();
        continue;
      } else if ((candidate != null) && (candidate.getPolicyWeight() == 0)) {
        candidate = candidate.getPreviousInAccessOrder();
        candidates--;
        continue;
      }

      // Evict immediately if only one of the entries is present
      // 此时哪一个不为空，就立刻删除该数据。
      if (victim == null) {
        @SuppressWarnings("NullAway")
        Node<K, V> previous = candidate.getPreviousInAccessOrder();
        Node<K, V> evict = candidate;
        candidate = previous;
```

```
        candidates--;
        evictEntry(evict, RemovalCause.SIZE, 0L);
        continue;
      } else if (candidate == null) {
        Node<K, V> evict = victim;
        victim = victim.getNextInAccessOrder();
        evictEntry(evict, RemovalCause.SIZE, 0L);
        continue;
      }

      // Evict immediately if an entry was collected
      // 因为软引用，在GC的时候被清除，此时立刻清空队列该位置数据
      K victimKey = victim.getKey();
      K candidateKey = candidate.getKey();
      if (victimKey == null) {
        @NonNull Node<K, V> evict = victim;
        victim = victim.getNextInAccessOrder();
        evictEntry(evict, RemovalCause.COLLECTED, 0L);
        continue;
      } else if (candidateKey == null) {
        candidates--;
        @NonNull Node<K, V> evict = candidate;
        candidate = candidate.getPreviousInAccessOrder();
        evictEntry(evict, RemovalCause.COLLECTED, 0L);
        continue;
      }

      // Evict immediately if the candidate's weight exceeds the maximum
      // 如果这个node的权重大于该队列的设定最大值，放不下的节点直接处理掉
      if (candidate.getPolicyWeight() > maximum()) {
        candidates--;
        Node<K, V> evict = candidate;
        candidate = candidate.getPreviousInAccessOrder();
        evictEntry(evict, RemovalCause.SIZE, 0L);
        continue;
      }

      // Evict the entry with the lowest frequency
      candidates--;
      //根据节点的统计频率frequency来做比较，看看要处理掉victim还是candidate
      //admit是具体的比较规则，看下面
      //如果candidate胜出则淘汰victim
      if (admit(candidateKey, victimKey)) {
        Node<K, V> evict = victim;
        victim = victim.getNextInAccessOrder();
        evictEntry(evict, RemovalCause.SIZE, 0L);
        candidate = candidate.getPreviousInAccessOrder();
       //如果是victim胜出，则淘汰candidate
      } else {
        Node<K, V> evict = candidate;
        candidate = candidate.getPreviousInAccessOrder();
        evictEntry(evict, RemovalCause.SIZE, 0L);
      }
    }
  }
```

下面是根据频率具体淘汰数据的流程。

```
/**
 * Determines if the candidate should be accepted into the main space, as determined by its
 * frequency relative to the victim. A small amount of randomness is used to protect against hash
 * collision attacks, where the victim's frequency is artificially raised so that no new entries
 * are admitted.
 *
 * @param candidateKey the key for the entry being proposed for long term retention
 * @param victimKey the key for the entry chosen by the eviction policy for replacement
 * @return if the candidate should be admitted and the victim ejected
 */
@GuardedBy("evictionLock")
boolean admit(K candidateKey, K victimKey) {
  //分别获取victim和candidate的统计频率
  int victimFreq = frequencySketch().frequency(victimKey);
  int candidateFreq = frequencySketch().frequency(candidateKey);
  //谁大谁赢
  if (candidateFreq > victimFreq) {
    return true;
```

```
  //如果相等，candidate小于5都当输了
  } else if (candidateFreq <= 5) {
    // The maximum frequency is 15 and halved to 7 after a reset to age the history. An attack
    // exploits that a hot candidate is rejected in favor of a hot victim. The threshold of a warm
    // candidate reduces the number of random acceptances to minimize the impact on the hit rate.
    return false;
  }
  //如果相等且candidate大于5，则随机淘汰一个
  int random = ThreadLocalRandom.current().nextInt();
  return ((random & 127) == 0);
}
```

二、caffeine高性能设计---MpscGrowableArrayQueue

第五行的代码是将实际写入的数据写入buffer池中，也就是caffeine的WAL机制的实现，其中writerBuffer是一个MpscGrowableArrayQueue队列。

该Mpsc的主要特征是 插入实现原子插入，**并支持多个生产者并发插入，但只有一个消费者单线程消费数据**。

```
1
2   void afterWrite(Runnable task) {
3       if (this.buffersWrites()) {
4           for(int i = 0; i < 100; ++i) {
5               // 尝试将写操作的数据任务task，放入buffer池中，如果满了，就放入失败
6               if (this.writeBuffer().offer(task)) {
7                   this.scheduleAfterWrite();
8                   return;
9               }
10              this.scheduleDrainBuffers();
11          }
12
13          try {
14              this.performCleanUp(task);
15          } catch (RuntimeException var3) {
16              logger.log(Level.SEVERE, "Exception thrown when performing the maintenance task", var3);
17          }
18      } else {
19          this.scheduleAfterWrite();
20      }
21
    }
```

下面为队列初始化的关键参数配置

capacity：初始化的size为2的整数倍

producerbuffer：生产者的数组指向初始化数组

consumerBuffer：消费者的数组指向初始化数组

```
Params: initialCapacity – the queue initial capacity. If chunk size is fixed this will be the chunk size.
        Must be 2 or more.
BaseMpscLinkedArrayQueue(final int initialCapacity) {
  if (initialCapacity < 2) {
    throw new IllegalArgumentException("Initial capacity must be 2 or more");
  }

  int p2capacity = ceilingPowerOfTwo(initialCapacity);
  // leave lower bit of mask clear
  long mask = (p2capacity - 1L) << 1;
  // need extra element to point at next array
  E[] buffer = allocate( capacity: p2capacity + 1);
  producerBuffer = buffer;
  producerMask = mask;
  consumerBuffer = buffer;
  consumerMask = mask;
  soProducerLimit(mask); // we know it's all empty to start with
}
```

下面是offer的具体方法，支持多线程的调用通过CAS来解决冲突。

```
@Override
 @SuppressWarnings("MissingDefault")
 public boolean offer(final E e) {
   if (null == e) {
     throw new NullPointerException();
   }
```

```
    long mask;
    E[] buffer;
    long pIndex;

    while (true) {
      long producerLimit = lvProducerLimit();
      pIndex = lvProducerIndex();
      // lower bit is indicative of resize, if we see it we spin until it's cleared
      // 最低位为1说明在resize，需要等resize结束
      if ((pIndex & 1) == 1) {
        continue;
      }
      // pIndex is even (lower bit is 0) -> actual index is (pIndex >> 1)

      // mask/buffer may get changed by resizing -> only use for array access after successful CAS.
      mask = this.producerMask;
      buffer = this.producerBuffer;
      // a successful CAS ties the ordering, lv(pIndex)-[mask/buffer]->cas(pIndex)

      // assumption behind this optimization is that queue is almost always empty or near empty
      // 生产者的位置达到了数组的“末端”，此时无法写入数据需要进行ji
      if (producerLimit <= pIndex) {
        int result = offerSlowPath(mask, pIndex, producerLimit);
        switch (result) {
          case 0:
            break;
          // RETRY, 重试
          case 1:
            continue;
          // 生产者buffer满就返回失败
          case 2:
            return false;
          // 需要扩容
          case 3:
            resize(mask, buffer, pIndex, e);
            return true;
        }
      }

      if (casProducerIndex(pIndex, pIndex + 2)) {
        break;
      }
    }
    // INDEX visible before ELEMENT, consistent with consumer expectation
    final long offset = modifiedCalcElementOffset(pIndex, mask);
    soElement(buffer, offset, e);
    return true;
  }
```

下面是poll函数，也就是buffer写入cache的实际写入线程，该方法标注了只能供一个线程调用。

```
/**
 * {@inheritDoc}
 * <p>
 * This implementation is correct for single consumer thread use only.
 */
@Override
@SuppressWarnings("unchecked")
public E poll() {
  final E[] buffer = consumerBuffer;
  final long index = consumerIndex;
  final long mask = consumerMask;

  final long offset = modifiedCalcElementOffset(index, mask);
  Object e = lvElement(buffer, offset);// LoadLoad
  if (e == null) {
    if (index != lvProducerIndex()) {
      // poll() == null iff queue is empty, null element is not strong enough indicator, so we
      // must
      // check the producer index. If the queue is indeed not empty we spin until element is
      // visible.
      do {
        e = lvElement(buffer, offset);
      } while (e == null);
    } else {
      return null;
```

```
    }
  }
  if (e == JUMP) {
    final E[] nextBuffer = getNextBuffer(buffer, mask);
    return newBufferPoll(nextBuffer, index);
  }
  soElement(buffer, offset, null);
  soConsumerIndex(index + 2);
  return (E) e;
}
```

无标签